[Company Name]

Development of an Intelligent RAG-Based Chatbot

Internship Report

Oumayma Chtioui

[Date]

DEDICATION

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Maecenas porttitor congue massa. Fusce posuere, magna sed pulvinar ultricies, purus lectus malesuada libero, sit amet commodo magna eros quis urna.

Nunc viverra imperdiet enim. Fusce est. Vivamus a tellus.

ACKNOWLEDGMENTS

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Maecenas porttitor congue massa. Fusce posuere, magna sed pulvinar ultricies, purus lectus malesuada libero, sit amet commodo magna eros quis urna.

ABSTRACT

This report presents the development of an intelligent chatbot based on RAG (Retrieval-Augmented Generation) architecture for Elyos Digital. The project aims to address the challenge of efficiently exploiting internal document repositories by providing precise, contextual, and verifiable responses in natural language while minimizing hallucination risks.

TABLE OF CONTENTS

TABLE DES MATIERES

Tapez le titre du chapitre (niveau 1) 1
Tapez le titre du chapitre (niveau 2) 2
Tapez le titre du chapitre (niveau 3) 3
Tapez le titre du chapitre (niveau 1) 4
Tapez le titre du chapitre (niveau 2) 5
Tapez le titre du chapitre (niveau 3) 6

LIST OF FIGURES

LIST OF TABLES

Table 1: Comparison of Existing Solutions

Table 2: Functional Requirements

Table 3: Non-Functional Requirements

Table 4: Development Tools and Technologies

GENERAL INTRODUCTION

In the current era of digital transformation, organizations generate and store an increasingly large volume of technical and operational documents. Although these repositories represent valuable sources of knowledge, accessing relevant information efficiently remains a significant challenge for employees. Traditional document search tools mainly rely on keyword-based queries, which often require precise knowledge of the document structure and fail to capture the semantic context of a user's request.

At the same time, recent advances in artificial intelligence, particularly in the field of Large Language Models (LLMs), have enabled the development of intelligent conversational systems capable of understanding and generating natural language. Despite their impressive capabilities, these models have certain limitations. Their knowledge is typically limited to their training data and they may sometimes produce inaccurate or unverifiable information when responding to user queries.

To address these limitations, the **Retrieval-Augmented Generation (RAG)** architecture has emerged as an effective approach that combines information retrieval from a document repository with the generative capabilities of language models. By integrating these two components, RAG-based systems can provide context-aware responses grounded in real documents, while reducing the risk of hallucinations and improving the reliability of generated answers.

This work is carried out as part of a **Final Studies Project (PFE)** for obtaining a bachelor's degree in Computer Science, conducted within the host organization **Elyos Digital**, a digital agency specialized in developing innovative web solutions and supporting organizations in their digital transformation. The main objective of this project is to design and implement an intelligent chatbot application based on the RAG architecture that allows users to query internal document repositories through natural language interactions.

The developed application aims to facilitate access to information, improve knowledge utilization within organizations, and provide accurate and contextual responses supported by document sources. It also seeks to offer a user-friendly interface for document management and conversation handling while ensuring performance, reliability, and security.

To effectively manage the development process, the **Scrum agile methodology** was adopted. This methodology promotes iterative development through short development cycles, allowing continuous improvement, rapid feedback, and better adaptation to evolving project requirements.

This report is structured into four chapters. The first chapter presents the general framework of the project and introduces the host organization, the project context, and its objectives. The second chapter reviews the state of the art related to conversational systems, large language models, and RAG architectures. The third chapter describes the system design and the technical architecture adopted for the proposed solution. Finally, the fourth chapter presents the implementation of the application. The report concludes with a general conclusion summarizing the results achieved and outlining potential future improvements.

CHAPTER 1

GENERAL FRAMEWORK AND PROJECT STUDY

INTRODUCTION

This chapter presents the general framework of the intelligent chatbot development project based on RAG architecture. We will begin with a detailed presentation of the host organization, Elyos Digital, before presenting the project context, problem statement, and objectives. We will then conduct a study of existing solutions, present our proposed solution, and detail the adopted methodology. Finally, we will analyze the functional and non-functional requirements, present the work environment and tools, and describe the software architecture.

1. PRESENTATION OF THE HOST ORGANIZATION

1.1 COMPANY IDENTITY

Company Name: Elyos Digital

Business Sector: Multidisciplinary web agency with a strong web platform development component to support clients' digital transformation.

Global Services:

- Digital strategy consulting
- Interface design and conception (UI/UX)
- Web and mobile development (Full Stack)
- Integration of artificial intelligence solutions
- Application hosting and maintenance

Location: Mohamed Shim Street, Rbat Monastir 5000, Tunisia

1.2 VISION AND POSITIONING

Elyos Digital has positioned itself as an innovative player in the field of digital transformation in Tunisia. The company supports its clients in their digital evolution by offering modern technological solutions adapted to the specific needs of each organization.

Core Values:

- **Innovation:** Proactive adoption of emerging technologies (AI, Cloud, etc.)
- **Excellence:** Quality and rigor in every delivery

- **Collaboration:** Close working relationship with clients
- **Agility:** Rapid adaptation capacity to market changes

2. Project Presentation

2.1 Project Framework

In a context where artificial intelligence is profoundly transforming interactions between companies and users, Elyos Digital aims to develop strong expertise in RAG (Retrieval-Augmented Generation) technologies to offer clients intelligent and contextual chatbot solutions.

This project is part of a dual strategic approach: acquiring cutting-edge expertise in conversational AI technologies, and creating a mature and marketable solution for clients facing internal documentation access challenges.

2.2 Problem Statement

How can we develop an intelligent system capable of effectively exploiting an internal document base to provide precise, contextual, and verifiable answers to questions posed in natural language, while minimizing hallucination risks and ensuring information traceability?

2.3 Project Objectives

Technical Objectives:

- Master complete RAG pipeline implementation
- Ensure optimal performance (response time < 5 seconds)
- Guarantee response reliability (Answer Relevancy > 0.8, Faithfulness > 0.9)
- Control operating costs (< €1000/month)

Functional Objectives:

- Provide an intuitive user experience
- Enable simple document management via admin interface
- Maintain conversation history with systematic source citations
- Support per-conversation document isolation

Strategic Objectives:

- Develop a competitive advantage in conversational AI
- Create a marketable product for target clients
- Build reusable expertise for future projects

3. Study of Existing Solutions

3.1 Existing Solutions

Traditional Document Search Systems:

- Elasticsearch: Full-text search engine based on keyword indexing
- Apache Solr: Open-source search platform with faceted navigation
- SharePoint Search: Microsoft's enterprise search solution

Conventional Chatbots:

- Dialogflow: Google's intent-based chatbot platform
- Microsoft Bot Framework: Enterprise bot development platform
- IBM Watson Assistant: AI-powered virtual agent

RAG-Based Solutions:

- ChatGPT with Plugins: OpenAI's retrieval plugin for document access
- Notion AI: Q&A over workspace documents
- Glean: Enterprise search with AI-powered answers

3.2 Critique of Existing Solutions

Limitations of Traditional Search:

- Requires precise keyword formulation
- No natural language understanding
- Returns document lists rather than direct answers
- Poor handling of semantic similarity

Limitations of Conventional Chatbots:

- Generic responses not grounded in company documents
- Limited to pre-defined intents and responses
- Expensive and time-consuming to maintain
- No source attribution or verification

Limitations of Existing RAG Solutions:

- High costs for commercial solutions (€50-200/user/month)
- Data privacy concerns with cloud-based LLMs
- Limited customization and control
- No per-conversation document isolation

4. Proposed Solution

To address the identified limitations, we propose the development of an intelligent chatbot based on RAG architecture, specifically designed for querying and exploiting internal document repositories with per-conversation document isolation.

Key Differentiators:

- **Per-Conversation Document Management:** Each conversation has its own isolated document set, ensuring context relevance and privacy
- **Hybrid Deployment:** Support for both cloud LLMs (OpenAI, Anthropic) and local models (Ollama)
- **Complete Control:** Full ownership of infrastructure and data
- **Cost-Effective:** Optimized architecture with budget controls (< €1000/month)

Solution Architecture Components:

- Complete RAG Pipeline with document ingestion, chunking, vectorization, and retrieval
- React + TypeScript frontend with real-time streaming
- FastAPI backend with JWT authentication
- PostgreSQL for structured data (users, sessions)
- MongoDB for document metadata and session-document mapping
- FAISS for vector storage and semantic search
- LangChain for RAG orchestration

5. Adopted Methodology **Can be changed**

For this project, we have adopted the **Scrum methodology**, an agile framework well-suited for software development projects with evolving requirements.

Scrum Principles Applied:

- **2-Week Sprints:** Iterative development cycles for continuous delivery
- **Daily Stand-ups:** Quick team synchronization meetings
- **Sprint Planning:** Backlog prioritization and task assignment
- **Sprint Review:** Demo and stakeholder feedback
- **Sprint Retrospective:** Continuous improvement reflection

Development Phases:

- Sprint 1-2: Project setup, authentication, and basic infrastructure
- Sprint 3-4: Document ingestion pipeline and vector storage
- Sprint 5-6: RAG implementation and LLM integration
- Sprint 7-8: Frontend development and per-session documents
- Sprint 9-10: Testing, optimization, and deployment

6. Requirements Analysis

6.1 Functional Requirements **Can be changed**

FR1 - User Authentication:

- User registration with email and password
- Login with JWT token generation
- Session management with automatic expiration

FR2 - Document Management:

- Upload documents (PDF, DOCX, TXT, MD) via drag-and-drop
- Add URLs for web content scraping
- View uploaded documents with status (processing/ready)
- Delete documents from the system
- Assign documents to specific conversation sessions

FR3 - Conversation Management:

- Create new conversation with uploaded documents
- View conversation history in sidebar
- Switch between different conversations
- Delete conversations
- Per-conversation document isolation

FR4 - Chat Interaction:

- Send questions in natural language
- Receive AI-generated responses with source citations
- Real-time response streaming
- View source document references for each response
- Add more documents during active conversation

FR5 - RAG Pipeline:

- Automatic text extraction from uploaded documents
- Document chunking with configurable size and overlap
- Generate embeddings for semantic search
- Vector similarity search in FAISS
- Context-aware response generation

6.2 Non-Functional Requirements **Can be changed**

NFR1 - Performance:

- Response time < 5 seconds for RAG queries
- First Contentful Paint (FCP) < 1.5 seconds
- Support 50+ concurrent users
- Document processing time < 2 minutes for 10MB files

NFR2 - Reliability:

- Answer Relevancy > 0.8
- Faithfulness (grounding in documents) > 0.9
- System uptime > 99%
- Graceful fallback when LLM unavailable

NFR3 - Security:

- JWT-based authentication on all endpoints
- Password hashing with bcrypt
- User data isolation (cannot access other users' documents)
- Input validation and sanitization
- Rate limiting on API endpoints

NFR4 - Scalability:

- Horizontal scaling capability for backend services
- Database connection pooling
- Caching layer with Redis (optional)
- Support for growing document repository

NFR5 - Usability:

- Intuitive interface requiring no training
- Responsive design (mobile, tablet, desktop)
- Clear error messages and user feedback
- Accessibility standards compliance (WCAG 2.1)

NFR6 - Cost Control:

- Total operating cost < €1000/month
- LLM token usage optimization
- Infrastructure cost monitoring

7. Work Environment and Tools

Frontend Development:

- React 18+ with TypeScript
- Vite for build tooling
- Tailwind CSS for styling
- React Router for navigation
- Axios for HTTP requests

Backend Development:

- Python 3.11+

- FastAPI framework
- Pydantic for data validation
- SQLAlchemy ORM
- PyJWT for authentication

RAG & AI Stack:

- LangChain for RAG orchestration
- HuggingFace Transformers for embeddings (all-MiniLM-L6-v2)
- FAISS for vector storage
- Ollama for local LLM inference (development)
- OpenAI/Anthropic APIs for production LLMs

Databases:

- PostgreSQL 15+ (users, sessions)
- MongoDB 6+ (document metadata)
- Redis 7+ (caching, optional)

Development Tools:

- VS Code with extensions (ESLint, Prettier, Python)
- Git for version control
- Docker for containerization
- Postman for API testing

Deployment & DevOps:

- Docker Compose for local development
- Vercel for frontend hosting
- Railway/Render for backend hosting
- GitHub Actions for CI/CD

8. Software Architecture

This system follows a layered architecture pattern, separating concerns across presentation, application, intelligence, and data layers. This architecture ensures scalability, maintainability, and clear separation of responsibilities.

Architecture Layers: Can be changed

1. Presentation Layer (Frontend)

- React application with TypeScript
- Components: AuthPage, UploadPage, ChatPage, Sidebar
- State management with React hooks
- Real-time updates via WebSocket/SSE

2. Application Layer (API Gateway)

- FastAPI server with RESTful endpoints
- JWT authentication middleware
- Request validation and error handling
- Rate limiting and security controls

3. Intelligence Layer (RAG Pipeline)

- LangChain orchestration
- Document processing (extraction, chunking)
- Embedding generation (HuggingFace)
- Vector search and retrieval
- LLM generation (OpenAI/Anthropic/Ollama)
- Confidence scoring and hallucination detection

4. Data Layer

- PostgreSQL: Users, sessions, conversation metadata
- MongoDB: Document metadata, session-document mappings
- FAISS: Vector embeddings for semantic search
- File System: Document storage

Key Architectural Patterns:

- **Repository Pattern:** Data access abstraction for databases
- **Service Layer:** Business logic separation from controllers
- **Dependency Injection:** Loose coupling and testability
- **Event-Driven:** WebSocket for real-time updates

Security Architecture:

- JWT authentication on all protected endpoints
- User data isolation at database level
- Session-based document access control
- Input validation and sanitization
- Rate limiting per user and endpoint

CONCLUSION

This chapter has established the comprehensive foundation of our intelligent RAG-based chatbot project. We have presented Elyos Digital as the host organization, defined the project framework and objectives, conducted a thorough study of existing solutions with their limitations, and proposed our innovative solution featuring per-conversation document isolation.

The adoption of Scrum methodology ensures an agile and iterative development approach. The detailed requirements analysis has identified both functional and non-functional needs, while the presentation of our work environment, tools, and software architecture provides a clear technical roadmap.

The following chapter will present a detailed state of the art, examining existing RAG implementations, LLM technologies, and vector database solutions to position our approach within the current technological landscape.

CONCLUSION

This Final Year Project was conducted within Elyos Digital, a company specialized in digital transformation and the development of innovative technological solutions. The project was carried out in a context where organizations manage large volumes of internal documentation, making the rapid retrieval of relevant information increasingly difficult. The main objective of this work was to design and develop an intelligent chatbot based on the Retrieval-Augmented Generation (RAG) architecture capable of querying internal document repositories and generating contextualized responses in natural language.

The proposed solution addresses the limitations of traditional document search systems and conventional chatbots. By combining semantic document retrieval with the generative capabilities of large language models, the developed application enables users to obtain precise, contextual, and verifiable answers based on the organization's documents. The integration of source citation, document management features, and conversation handling contributes to improving knowledge accessibility and reducing the time required to locate relevant information.

The development of this application followed the Scrum agile methodology, which enabled an iterative and flexible development process. Through short development cycles and continuous testing, the system was progressively improved while ensuring that the implemented features aligned with the project objectives. This approach facilitated the integration of different components of the RAG pipeline, including document processing, vector embedding generation, semantic search, and response generation.

From a technical perspective, the developed system integrates modern technologies such as FastAPI for the backend, React with TypeScript for the frontend, and FAISS for vector similarity search. These components collectively ensure a scalable architecture capable of efficiently processing user queries while maintaining acceptable performance and response times. The application also provides essential functionalities such as document upload, conversation management, and natural language interaction, which contribute to a user-friendly experience.

The project also provides practical value for the host organization. By exploring and implementing RAG-based conversational systems, Elyos Digital strengthens its expertise in artificial intelligence and conversational technologies. The solution developed during this project can serve as a foundation for future intelligent knowledge management systems and can potentially be adapted for different clients facing similar documentation access challenges.

Despite the promising results obtained, several improvements can be considered for future development. These include enhancing document processing capabilities, integrating more advanced evaluation metrics to measure response accuracy, improving scalability for larger document repositories, and supporting additional file formats and multilingual interactions. Further work could also explore the integration of more advanced local language models to improve data privacy and reduce operational costs.

In conclusion, this project has provided valuable practical experience in the design and implementation of modern artificial intelligence systems. It allowed the application of theoretical knowledge in machine learning, software architecture, and web development to a real-world problem. Beyond its technical contributions, this work represents an important step in the development of professional skills and demonstrates the potential of RAG-based conversational systems in improving knowledge accessibility within organizations.